

SO-101 主臂控制松灵 Piper 机械臂 ACT 模型 实验手册

示教数据采集 - AutoDL 云端训练 - 本地部署推理

【实验说明】 本实验使用 SO-101 leader 作为主操作臂，通过 CAN 总线控制松灵 Piper 机械臂完成示教数据采集，并在 AutoDL 云端训练 ACT 模型，最后下载模型在本地进行真实机器人部署。示例任务为 Put the banana on the plate.；如果实际任务不同，需要在数据采集命令和部署命令中同步修改任务文本，并建议同步修改数据集名称。AutoDL 的 SSH 地址、端口和用户名均以每位同学自己的实例信息为准。

适用对象：已经完成 LeRobot + SO-101 基础环境、SO-101 leader 校准、基础遥操作和摄像头识别的实验人员。

完成标准：能够使用 SO-101 leader 遥操作 Piper 采集 50 条左右示教数据，将数据上传到 AutoDL 云端完成 ACT 训练，并下载 checkpoint 在本地运行 Piper 推理部署。

目录

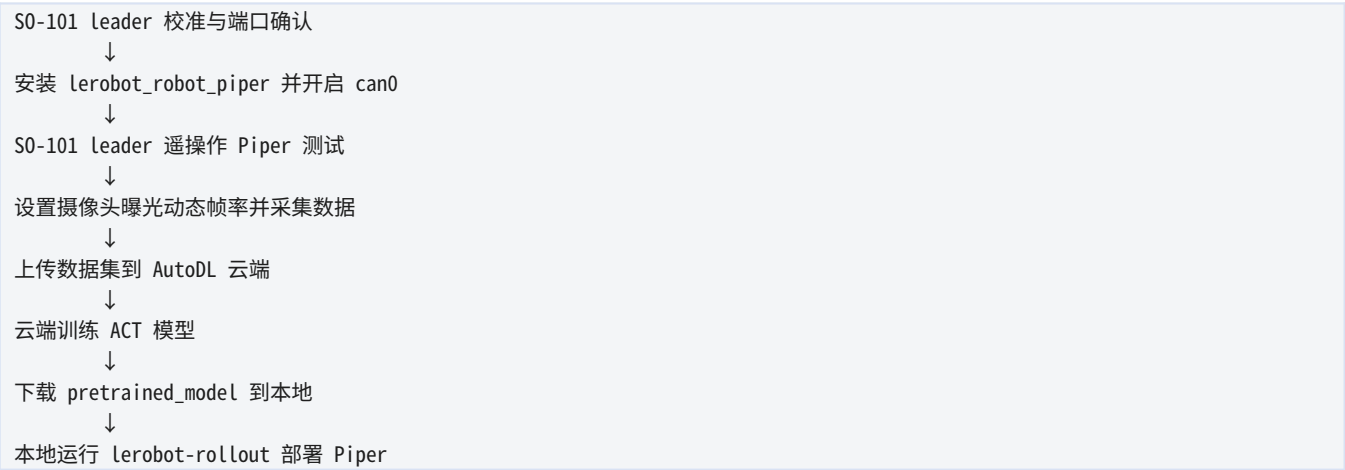
- 1. 实验目标与整体流程
- 2. 系统组成与安全要求
- 3. Piper 支持安装与 CAN 接口准备
- 4. SO-101 主臂控制 Piper 遥操作测试
- 5. Piper 示教数据采集
- 6. 数据上传至 AutoDL 云端
- 7. 云端 ACT 模型训练
- 8. 模型下载与本地整理
- 9. 本地部署推理与问题排查

1. 实验目标与整体流程

本实验目标是打通“SO-101 主臂遥操作 Piper - 真实示教数据采集 - 云端 ACT 模型训练 - 本地真实机器人部署”的完整流程。与 SO-101 follower 实验不同，本实验中 SO-101 只作为主操作臂，真实执行端是松灵 Piper 机械臂。

阶段	主要操作	完成标志
本地准备	安装 Piper 支持包，开启 can0，识别 SO-101 leader 端口与摄像头。	SO-101 leader 能控制 Piper 运动。
数据采集	使用 lerobot-record 采集 Piper 执行任务的数据。	本地生成 piper_record_test 数据集。
云端训练	上传数据集到 AutoDL，在 GPU 服务器上训练 ACT。	输出目录下生成 pretrained_model。
本地部署	下载模型，使用 lerobot-rollout 控制 Piper 执行任务。	Piper 能根据策略产生接近示教的动作。

整体流程如下：



2. 系统组成与安全要求

本实验由 SO-101 leader、Piper 机械臂、CAN 通信设备、摄像头、本地电脑和 AutoDL 云服务器组成。SO-101 leader 负责产生人工示教输入，Piper 机械臂负责真实执行动作，摄像头负责采集视觉观测。

组成	作用	注意事项
SO-101 leader	人工遥操作输入设备。	需要提前完成校准，并确认串口如 /dev/ttyACM0。
松灵 Piper 机械臂	真实执行端。	通过 CAN 接口通信，通常使用 can0。
CAN 设备	连接电脑与 Piper。	bitrate 需要设置为 1000000。
front 摄像头	观察整体操作场景。	示例使用 index_or_path: 0。
wrist 摄像头	观察腕部或末端附近场景。	示例使用 index_or_path: 2。
AutoDL 云服务器	用于 ACT 模型训练。	只需要数据集和 GPU，不需要连接机械臂。

【关键区别】 本实验没有使用 SO-101 follower。不要把 SO-101 follower 的串口命令直接套到 Piper 实验中；Piper 的控制接口是 can0，而 SO-101 leader 仍然使用串口。

- 首次开启 Piper 或运行策略部署前，应确保机械臂运动范围内没有人手、线材和易碎物体。
- 遥操作和部署时都应准备随时按 Ctrl+C 停止程序；必要时直接断电。
- 示教数据采集时摄像头位置不要移动；部署时摄像头位置应尽量与采集时一致。
- 如果任务不是示例中的“把香蕉放到盘子上”，需要在采集和部署两处同步修改任务文本。

3. Piper 支持安装与 CAN 接口准备

3.1 进入本地 LeRobot 环境

打开本地 Ubuntu 终端，进入已经搭建好的 LeRobot 环境。

```
conda activate lerobot
```

3.2 安装 Piper 支持包

Piper 支持包提供 LeRobot 与松灵 Piper 机械臂之间的适配代码。第一次实验时执行下面命令安装。

```
git clone https://github.com/AgRoboticsResearch/lerobot_robot_piper && pip install -e ./lerobot_robot_piper
```

【目录已存在时】 如果已经克隆过 lerobot_robot_piper，再次执行 git clone 可能提示目录已存在。此时可以进入该目录执行 git pull，然后重新 pip install -e .。

```
cd lerobot_robot_piper
git pull
pip install -e .
```

3.3 开启 can0 接口

Piper 通过 CAN 总线通信。每次重新插拔 CAN 设备、重启电脑或重新开机后，通常都需要重新开启 can0。

```
sudo ip link set can0 type can bitrate 1000000 && sudo ip link set can0 up
```

开启后检查 can0 状态：

```
ip -details link show can0
```

如果 can0 正常，应能看到接口状态为 UP，并显示 bitrate 1000000。

【CAN 接口排查】 如果提示 Cannot find device "can0"，说明系统没有识别到 CAN 设备，需要检查 USB-CAN 适配器、驱动、线材连接和设备名称。有些设备可能不是 can0，需要用 ip link 查看实际接口名。

4. SO-101 主臂控制 Piper 遥操作测试

4.1 确认 SO-101 leader 端口

连接 SO-101 leader 的 USB，并确认控制板已经上电。执行端口识别命令：

```
lerobot-find-port
```

假设识别到 SO-101 leader 端口为 /dev/ttyACM0，则授权该端口：

```
sudo chmod 666 /dev/ttyACM0
```

【端口注意】 /dev/ttyACM0 只是示例。若你的 leader 端口是 /dev/ttyACM1 或 /dev/ttyUSB0，需要把后续命令中的 --teleop.port 同步改为实际端口。

4.2 测试 SO-101 遥操作 Piper

遥操作测试用于确认 SO-101 leader、Piper、CAN 接口和 LeRobot Piper 支持包均能正常工作。第一次测试时不接入摄像头，降低排查难度。

```
lerobot-teleoperate \
  --robot.type=lerobot_robot_piper \
  --robot.can_interface=can0 \
  --robot.bitrates=1000000 \
  --robot.include_gripper=true \
  --robot.use_degrees=false \
  --teleop.type=so101_leader \
  --teleop.id=so101_leader_01 \
  --teleop.port=/dev/ttyACM0 \
  --teleop.use_degrees=false \
  --robot.cameras='{'
```

测试时缓慢移动 SO-101 leader，观察 Piper 是否平稳跟随。先测试单个关节，再测试夹爪和多关节组合。

- Piper 能随 SO-101 leader 运动，说明基本通信链路正常。
- 若 Piper 不动，优先检查 can0 是否 UP、Piper 是否上电、leader 端口是否正确。
- 若方向或幅度明显异常，先停止程序，再检查 leader 校准和 use_degrees 参数是否与命令一致。

5. Piper 示教数据采集

5.1 检查摄像头编号

数据采集需要图像输入。示例中使用 front 和 wrist 两路摄像头，编号分别为 0 和 2。实际编号需要先检查。

```
v4l2-ctl --list-devices
ls -l /dev/video*
lerobot-find-cameras opencv
```

如果不能确定摄像头编号，也可以用 OpenCV 逐个测试：

```
python - <<'EOF'
import cv2
for i in range(6):
    cap = cv2.VideoCapture(i)
    ok = cap.isOpened()
    ret, frame = cap.read() if ok else (False, None)
    print(i, 'opened=', ok, 'read=', ret, 'shape=', None if frame is None else frame.shape)
    cap.release()
EOF
```

5.2 设置摄像头曝光动态帧率

最新实验流程中增加了摄像头曝光动态帧率设置，用于提高数据采集频率的稳定性。示例中 front 摄像头对应 /dev/video0，wrist 摄像头对应 /dev/video2。

```
v4l2-ctl -d /dev/video0 -c exposure_dynamic_framerate=0
v4l2-ctl -d /dev/video2 -c exposure_dynamic_framerate=0
```

【摄像头注意】 这里的 /dev/video0 和 /dev/video2 需要与实际摄像头对应。如果摄像头不支持 exposure_dynamic_framerate，命令可能提示 unknown control；此时可先跳过该设置，但需要在采集时观察实际 FPS 是否稳定。

5.3 正式采集数据

确认 Piper 遥操作正常、摄像头编号正确、can0 已开启后，开始采集示教数据。示例保存路径为 \$HOME/lerobot_work/data/piper_record_test。

```
mkdir -p $HOME/lerobot_work/data/piper_record_test

lerobot-record \
  --robot.type=piper \
  --robot.can_interface=can0 \
  --robot.bitrate=1000000 \
  --robot.include_gripper=true \
```

```
--robot.use_degrees=false \
--robot.cameras="{front: {type: opencv, index_or_path: 0, width: 640, height: 480, fps: 30},wrist: {type: opencv,
index_or_path: 2, width: 640, height: 480, fps: 30}}" \
--teleop.type=so101_leader \
--teleop.port=/dev/ttyACM0 \
--teleop.id=so101_leader_01 \
--dataset.repo_id=local/piper_record_test \
--dataset.root=$HOME/lerobot_work/data/piper_record_test \
--dataset.push_to_hub=False \
--dataset.num_episodes=50 \
--dataset.single_task="Put the banana on the plate." \
--dataset.episode_time_s=30 \
--dataset.reset_time_s=15 \
--dataset.streaming_encoding=true \
--dataset.encoder_threads=2 \
--dataset.fps=15 \
--dataset.rgb_encoder.crf=20
```

【任务文本注意】 如果实际任务不是 Put the banana on the plate., 需要修改 `--dataset.single_task`, 并建议把 `piper_record_test` 改成与任务对应的数据集名称, 避免不同任务的数据混在一起。后续训练和部署命令中的 `dataset.repo_id`、`dataset.root`、`policy.path` 和 `--task` 也应保持对应关系。

参数	含义
<code>--robot.type=piper</code>	指定执行端为 Piper 机械臂。
<code>--robot.can_interface=can0</code>	使用 can0 与 Piper 通信。
<code>--robot.include_gripper=true</code>	采集和控制夹爪。
<code>--robot.cameras</code>	声明 front 与 wrist 两路摄像头。
<code>--teleop.type=so101_leader</code>	使用 SO-101 leader 作为遥操作输入。
<code>--dataset.num_episodes=50</code>	采集 50 条 episode。
<code>--dataset.fps=15</code>	数据集保存帧率为 15 FPS。
<code>--dataset.rgb_encoder.crf=20</code>	视频编码质量参数, 数值越低通常质量越高、文件越大。

录制时常用按键: n 或右方向键结束当前 episode, r 或左方向键取消当前 episode 并重录, q 或 ESC 停止录制并保存已经完成的数据。失败、卡住、明显偏离任务目标的数据不要保留。

5.4 检查数据是否保存成功

```
du -sh $HOME/lerobot_work/data/piper_record_test
find $HOME/lerobot_work/data/piper_record_test -maxdepth 3 -type f | head -n 30
```

如果目录大小很小或没有文件, 说明数据没有正常保存, 需要重新采集。正式训练前建议先录制 1-2 条测试数据, 确认保存正常后再录制 50 条正式数据。

6. 数据上传至 AutoDL 云端

本节将本地采集的数据集上传到 AutoDL 服务器。示例命令中的端口号、服务器地址和用户名仅作为示例, 实际使用时必须替换为每位同学自己 AutoDL 实例页面显示的信息。

6.1 上传数据集

在本地电脑终端执行：

```
scp -rP 38671 $HOME/lerobot_work/data/piper_record_test root@connect.westb.seetacloud.com:/root/
```

【AutoDL 连接注意】 scp 命令中端口参数是大写 -P；ssh 登录时通常是小写 -p。38671、connect.westb.seetacloud.com 和 root 都应以自己的 AutoDL 实例为准。

6.2 登录云端并检查数据

```
ssh -p 38671 root@connect.westb.seetacloud.com

ls /root/piper_record_test
du -sh /root/piper_record_test
find /root/piper_record_test -maxdepth 3 -type f | head -n 20
```

如果 /root/piper_record_test 不存在，需要检查上传命令中的本地路径、远程路径和 AutoDL 连接信息是否正确。

7. 云端 ACT 模型训练

7.1 云端环境检查与准备

云端只负责训练，不需要连接 Piper 或 SO-101。若云端已经按照前一次 ACT 实验配置好 lerobot 环境，可以直接跳到 7.2。若是新服务器，先执行以下命令。

```
conda init bash
source ~/.bashrc
conda create -n lerobot python=3.12 -y
conda activate lerobot
sudo apt update
sudo apt install -y ffmpeg
pip install --upgrade pip
pip install 'lerobot[training]'
```

检查 GPU 和 PyTorch：

```
nvidia-smi
python - <<'EOF'
import torch
print('cuda available:', torch.cuda.is_available())
if torch.cuda.is_available():
    print(torch.cuda.get_device_name(0))
EOF
```

7.2 进入 screen

训练时间较长，建议使用 screen 防止 SSH 断开导致终端关闭。

```
screen -S act_piper_train
conda activate lerobot
```

从 screen 中临时退出可以按 Ctrl+A，然后按 D；重新进入可执行 screen -r act_piper_train。

7.3 开始训练 ACT

```
lerobot-train \
  --dataset.root=/root/piper_record_test \
  --dataset.repo_id=local/piper_record_test \
  --policy.type=act \
  --output_dir=outputs/train/act_piper_test \
  --policy.device=cuda \
  --wandb.enable=false \
  --policy.push_to_hub=false
```

训练过程中可以新开一个 SSH 终端查看 GPU 使用情况：

```
watch -n 1 nvidia-smi
```

训练完成或训练到一定步数后，检查 checkpoint：

```
find /root/outputs/train/act_piper_test -type d -name "pretrained_model"
```

常见路径为 /root/outputs/train/act_piper_test/checkpoints/020000/pretrained_model。实际 checkpoint 编号可能不同，以 find 命令输出为准。

8. 模型下载与本地整理

训练后需要将云端 checkpoint 下载到本地电脑。示例下载 checkpoints/020000 下的 pretrained_model。

```
cd $HOME/lerobot_work
mkdir -p models/act_piper_test_20000

scp -rP 38671 \
  root@connect.westb.seetacloud.com:/root/outputs/train/act_piper_test/checkpoints/020000/pretrained_model \
  models/act_piper_test_20000/
```

检查本地模型路径：

```
find $HOME/lerobot_work/models/act_piper_test_20000 -maxdepth 3 -type f | head -n 30
```

【模型路径注意】若直接把远程 pretrained_model 下载为 act_piper_test_20000，可能导致本地目录层级不同。部署前应使用 find 命令确认实际 pretrained_model 路径，并将 --policy.path 改成真实路径。

9. 本地部署推理与问题排查

9.1 部署前检查

- Piper 已上电，CAN 线和电源线连接正常。
- can0 已开启，bitrate 为 1000000。
- front 和 wrist 摄像头编号与采集时一致。
- 物体初始位置、摄像头位置和光照尽量接近采集数据。
- 已下载 ACT 模型，并确认 --policy.path 指向 pretrained_model。
- 第一次部署建议 duration 设置为 10-20 秒，并随时准备 Ctrl+C。

部署前重新开启 can0，并可再次设置摄像头曝光动态帧率：

```
sudo ip link set can0 type can bitrate 1000000 && sudo ip link set can0 up
v4l2-ctl -d /dev/video0 -c exposure_dynamic_framerate=0
v4l2-ctl -d /dev/video2 -c exposure_dynamic_framerate=0
```

9.2 本地部署命令

下面命令假设当前目录为 \$HOME/lerobot_work，并且模型位于 models/act_piper_test_20000/pretrained_model。

```
cd $HOME/lerobot_work
conda activate lerobot

lerobot-rollout \
  --strategy.type=base \
  --policy.path=./models/act_piper_test_20000/pretrained_model \
  --robot.type=piper \
  --robot.can_interface=can0 \
  --robot.bitrate=1000000 \
  --robot.include_gripper=true \
  --robot.use_degrees=false \
  --robot.cameras="{front: {type: opencv, index_or_path: 0, width: 640, height: 480, fps: 30},wrist: {type: opencv, index_or_path: 2, width: 640, height: 480, fps: 30}}" \
  --task="Put the banana on the plate" \
  --duration=20
```

【部署任务注意】 如果训练任务不是 Put the banana on the plate，需要将部署命令中的 --task 改为与采集数据时 --dataset.single_task 对应的任务描述。摄像头名称 front、wrist 以及分辨率也应与采集阶段保持一致。

9.3 常见问题排查

现象	可能原因	处理方式
Cannot find device can0	CAN 设备未识别或接口名称不是 can0。	执行 ip link 查看接口；检查 USB-CAN 驱动和线材。
Piper 不动	can0 未 UP、Piper 未上电、bitrate 错误。	重新开启 can0，检查电源和 CAN 连接。
leader 无法读取	SO-101 leader 端口错误或无权限。	重新运行 lerobot-find-port，并

		chmod 666 实际端口。
摄像头打不开	index_or_path 写错或摄像头被占用。	运行 lerobot-find-cameras opencv, 修改 front/wrist 编号。
训练找不到数据集	云端路径不是 /root/piper_record_test。	在云端 find /root -maxdepth 3 -name piper_record_test。
policy.path 报错	本地模型目录层级不一致。	find \$HOME/lerobot_work -type d -name pretrained_model, 然后修改 --policy.path。
部署动作异常	数据质量差、摄像头位置变化、任务文本不一致或 checkpoint 不合适。	先确认遥操作正常, 再检查摄像头、任务文本和模型路径。

附：本实验关键命令清单

以下命令仅用于快速核对，正式实验时应按照前文步骤逐步执行。

```
# 安装 Piper 支持
conda activate lerobot
git clone https://github.com/AgRoboticsResearch/lerobot_robot_piper && pip install -e ./lerobot_robot_piper

# 开启 can0
sudo ip link set can0 type can bitrate 1000000 && sudo ip link set can0 up

# 遥操作测试
lerobot-teleoperate --robot.type=lerobot_robot_piper --robot.can_interface=can0 --robot.bitrate=1000000 --
robot.include_gripper=true --robot.use_degrees=false --teleop.type=so101_leader --teleop.id=so101_leader_01 --
teleop.port=/dev/ttyACM0 --teleop.use_degrees=false --robot.cameras='{}'

# 摄像头动态帧率设置
v4l2-ctl -d /dev/video0 -c exposure_dynamic_framerate=0
v4l2-ctl -d /dev/video2 -c exposure_dynamic_framerate=0

# 云端训练
lerobot-train --dataset.root=/root/piper_record_test --dataset.repo_id=local/piper_record_test --policy.type=act --
output_dir=outputs/train/act_piper_test --policy.device=cuda --wandb.enable=false --policy.push_to_hub=false
```